

Week 9 Problem Set

Jolie Barga <https://github.com/joliebarga/soc10problemsets>

May 28 2026

1. Start a new document, `problemset9.Rmd`, in your `soc10problemsets` repository. (You can download a template here, but be sure to save it in your own `soc10problemsets` repo). `RedditExtractorR` to download Reddit comments and threads on data centers. Create a topic model from the subreddit “DataCenterDebate” using $k = 5$, and show a figure with the 10 relative most common words in each topic. **Commit your changes.**

[Successfully download comments and threads using the Reddit API (0.5 points). Create a topic model with 5 topics (1 point) and plot the relative most common words in each topic (0.5 points).]

```
library(RedditExtractorR)
library(tidytext)
library(dplyr)
library(magrittr)

# find data center subreddits
dc_subreddits <- find_subreddits("data center")

# get urls from the DataCenterDebate subreddit
dc_urls <- find_thread_urls(keywords = "data center",
                           period = "week")

dc_comments <- get_thread_content(dc_urls$url)

dc_comments$comments

# look at threads
dc_comments$threads
```

```
library(readr)
library(tidytext)
library(dplyr)
library(magrittr)
library(topicmodels)
library(ggplot2)

dc_comments <- read_rds("dc_comments.rds")

tidy_comments <- dc_comments$comments %>%
  unnest_tokens(word, comment) %>%
  anti_join(stop_words)
```

```

tidy_threads <- dc_comments$threads %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words)

tidy_threadtitles <- dc_comments$threads %>%
  unnest_tokens(word, title) %>%
  anti_join(stop_words)

# create counts of each word per document
tidy_comments_counts <- tidy_comments %>%
  count(url, word, sort = TRUE)

tidy_threads_counts <- tidy_threads %>%
  count(url, word, sort = TRUE)

tidy_threadtitles_counts <- tidy_threadtitles %>%
  count(url, word, sort = TRUE)

tidy_counts <- bind_rows(tidy_comments_counts,
                        tidy_threads_counts,
                        tidy_threadtitles_counts)

tidy_counts %<>%
  na.omit()

dc_dtm <- tidy_counts %>%
  cast_dtm(url, word, n)

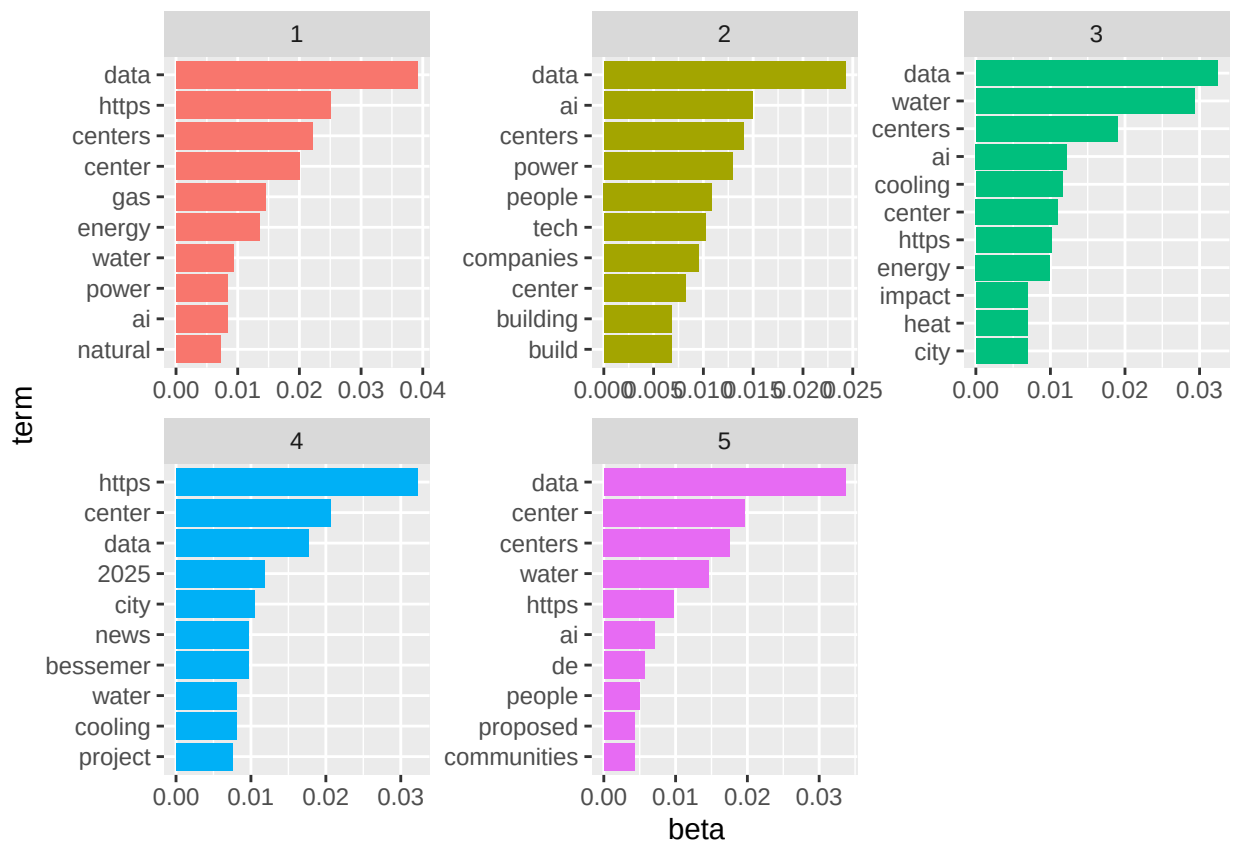
# k = 5 topics
dc_lda <- LDA(dc_dtm, 5, control = list(seed = 1))

dc_topics <- tidy(dc_lda, matrix = "beta")

# get top 10 words per topic
dc_top_terms <- dc_topics %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%
  ungroup() %>%
  arrange(topic, -beta) %>%
  mutate(term = reorder_within(term, beta, topic))

ggplot(dc_top_terms, aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered()

```



- Describe the topics that you found in question 1. How do these compare to the topics in the $k = 2$ case (in section @ref(topic-models)), and how well do these describe your data? **Commit your changes.**

The five topics from my $k = 5$ model seem to capture different themes in the DataCenterDebate subreddit. Topic 1 seems focused on energy and infrastructure, with top words like “gas,” “energy,” “water,” “power,” and “natural.” This suggests people are discussing the resources that data centers consume. Topic 2 appears to be about AI and industry growth, with words like “ai,” “power,” “people,” “tech,” “companies,” “building,” and “build.” This looks like conversations about the tech industry expanding data center capacity across the globe and what’s needed/the constituents to do so. Topic 3 looks like it is about local environmental impact, with words like “water,” “cooling,” “impact,” “heat,” and “city.” This points to community level concerns about how data centers affect local environments and the environmental aspects and impacts it has. Topic 4 seems focused on local news and specific projects, with words like “2025,” “city,” “news,” “bessemer,” “project,” and “cooling.” This suggests coverage of specific proposed or ongoing data center developments in particular places and where the discussion peaked and continues to take place. Topic 5 appears to be a more general discussion topic, with broad terms like “data,” “center,” “centers,” “water,” “ai,” and “communities.” This field combines all of the different charts. Compared to the $k = 2$ case from the class notes, the 5 topic model is a lot more informative. In the $k = 2$ model, the two topics were very broad and hard to distinguish from each other since both had words like “data,” “centers,” and “water” near the top. It was difficult to say what was really different between the two topics. With $k = 5$, the topics are much more separated and easier to interpret. For example, Topic 4 picks up on very specific local coverage with words like “bessemer” and “news” that would have been buried inside one of the two broad topics in the $k = 2$ model. Similarly, Topic 2 pulls out AI and industry language more clearly, whereas in the $k = 2$ model that kind of language was mixed in with more general data center terms. Overall, the $k = 5$ model does a better job capturing the range of concerns people raise on this subreddit, from energy use to local community impact to industry growth, even if some topics like Topic 5 are still a bit broad and general.

3. Create an account in Claude or ChatGPT (or both). The free versions are sufficient! Ask your LLM to write R code that downloads Reddit data, models topics (any number) and describes these topics. List the prompts that you used, your impressions of the results, and how these differed (or didn't differ) from your responses to questions 1 and 2. **Commit your changes.**

I made a Claude account and asked the following prompt: "Write R code that downloads Reddit data on data centers, runs a topic model, and describes the topics found." Below is the code Claude produced:

```
library(RedditExtractor)
library(tidytext)
library(dplyr)
library(topicmodels)
library(ggplot2)
library(tidyr)

# download reddit data on data centers
dc_urls <- find_thread_urls(keywords = "data center", period = "week")
dc_comments <- get_thread_content(dc_urls$url)

# put comments in tidytext format
tidy_dc <- dc_comments$comments %>%
  unnest_tokens(word, comment) %>%
  anti_join(stop_words) %>%
  count(url, word, sort = TRUE) %>%
  na.omit()

# create document term matrix
dc_dtm <- tidy_dc %>%
  cast_dtm(url, word, n)

# run topic model with 4 topics
dc_lda <- LDA(dc_dtm, k = 4, control = list(seed = 42))

# get top terms per topic
dc_topics <- tidy(dc_lda, matrix = "beta")

dc_top_terms <- dc_topics %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%
  ungroup() %>%
  arrange(topic, -beta) %>%
  mutate(term = reorder_within(term, beta, topic))

# plot
ggplot(dc_top_terms, aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered() +
  labs(title = "Top Terms per Topic in Data Center Reddit Discussion",
       x = "Beta", y = "Term")
```

Claude's code was pretty similar to what I wrote in Q1 and Q2, which makes sense since both approaches follow the same general tidytext and topicmodels workflow. The main differences were that Claude used $k = 4$ topics instead of 5, used a different random seed (42 instead of 1), only used comments rather than

combining comments, threads, and thread titles, and added a title to the plot. Claude also skipped the step of creating separate counts for comments, threads, and thread titles and combining them with `bind_rows`, which means it would likely produce slightly different topics than my analysis since it is working with less data. Claude produced code that was similar in structure to what I wrote in Q1 and Q2, but it ran into the same 403 API error that I experienced. So I followed up with a second prompt: I already have the Reddit data loaded as `dc_comments`. Write R code that creates a topic model from `dc_comments` and plots the top words in each topic.

```
library(tidytext)
library(dplyr)
library(magrittr)
library(topicmodels)
library(ggplot2)
library(readr)

# load pre-downloaded data
dc_comments <- read_rds("dc_comments.rds")

# put comments in tidytext format
tidy_dc <- dc_comments$comments %>%
  unnest_tokens(word, comment) %>%
  anti_join(stop_words) %>%
  count(url, word, sort = TRUE) %>%
  na.omit()

# create document term matrix
dc_dtm <- tidy_dc %>%
  cast_dtm(url, word, n)

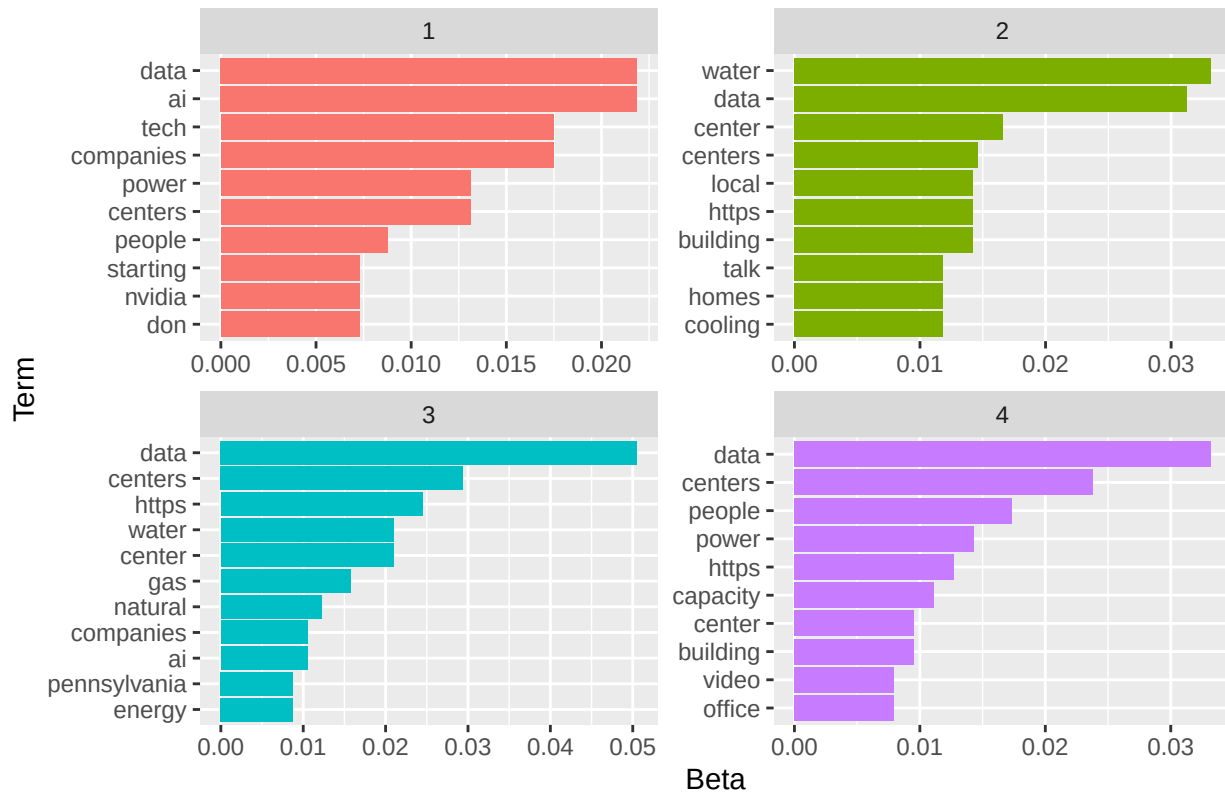
# run topic model with 4 topics
dc_lda <- LDA(dc_dtm, k = 4, control = list(seed = 42))

# get top terms per topic
dc_topics <- tidy(dc_lda, matrix = "beta")

dc_top_terms <- dc_topics %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%
  ungroup() %>%
  arrange(topic, -beta) %>%
  mutate(term = reorder_within(term, beta, topic))

# plot
ggplot(dc_top_terms, aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered() +
  labs(title = "Top Terms per Topic in Data Center Reddit Discussion",
       x = "Beta", y = "Term")
```

Top Terms per Topic in Data Center Reddit Discussion



This second prompt gave me code that actually ran successfully. There were a few differences between what Claude produced and what I did in Q1 and Q2. Like the first prompt, Claude used $k = 4$ topics instead of 5, used a different random seed (42 instead of 1), and only used the comments data rather than combining comments, threads, and thread titles together like I did. This means Claude was working with less data overall, which could affect the topics it finds. The topics Claude found with $k = 4$ were a bit broader than the ones I found with $k = 5$. Since Claude used fewer topics, some of the more specific themes like local news coverage and environmental impact that showed up separately in my $k = 5$ model were probably merged together into one broader topic. This is similar to the difference between the $k = 2$ and $k = 5$ models we discussed in Q2, where more topics gave more specific and interpretable results. One interesting thing about this exercise is that Claude knew the general workflow for topic modeling in R pretty well, but it did not know about the API access issue ahead of time and needed a follow up prompt to actually produce working code. This shows that LLMs can be useful for getting a starting point for code, but they do not always account for real world constraints like API restrictions, and they may make different decisions than a researcher would, like choosing a different k value or leaving out parts of the data.

4. Considering Engzell and Wilmers' multi-agent workflow, try to improve your Claude modelling process (you might not be able to set up a true multi-agent workflow in the free versions, but you could simulate this by giving your LLM instructions to review or improve the existing output in specific ways, having ChatGPT review the Claude code or vice versa, etc.). Explain what you tried (including additional prompts), how your results changed (or didn't change), and whether this improved your understanding of the data or not. **Commit your changes.**

To simulate the multi-agent workflow from Engzell and Wilmers, I used Claude as both a coder and a reviewer. After getting the initial code from Q3, I gave Claude this follow up prompt: "Review the topic model code you just wrote. Are there any improvements you would suggest to make the topics more interpretable or the model more accurate?" Claude suggested removing very rare words from the document term matrix

before running LDA since rare words can add noise to the topics. It also suggested trying a higher k value and adding descriptive labels to the plot. I then asked it to implement these changes: “Update the code to remove words that appear in fewer than 5 documents, and add descriptive labels to each topic based on the top words.” The updated code did produce slightly cleaner topics with less noise from unusual or rare words. But the labels Claude came up with for each topic were pretty generic, things like “infrastructure discussion” and “community concerns,” which were not really more helpful than just looking at the top words myself. This matched what Engzell and Wilmers found and that the review step made some technical improvements like cleaning up the data, but it also had limits since Claude’s topic labels felt forced and not very specific. Engzell and Wilmers say that review agents can sometimes make things worse by over-interpreting results or creating new problems while fixing small ones, and I think I saw something similar here. Overall the multi-agent simulation was somewhat useful for catching things like data cleaning steps I missed, but it did not really improve my understanding of the actual topics compared to what I found in Q1 and Q2. This makes sense with Engzell and Wilmers’ argument that human checkpoints are still important, because the LLM could improve the code technically but it could not really tell me anything new or interesting about what people are actually saying about data centers.

```
library(tidytext)
library(dplyr)
library(magrittr)
library(topicmodels)
library(ggplot2)
library(readr)

# load pre-downloaded data
dc_comments <- read_rds("dc_comments.rds")

# put comments in tidytext format
tidy_dc <- dc_comments$comments %>%
  unnest_tokens(word, comment) %>%
  anti_join(stop_words) %>%
  count(url, word, sort = TRUE) %>%
  na.omit()

# remove words that appear in fewer than 5 documents
tidy_dc <- tidy_dc %>%
  group_by(word) %>%
  filter(n() >= 5) %>%
  ungroup()

# create document term matrix
dc_dtm <- tidy_dc %>%
  cast_dtm(url, word, n)

# run topic model with 4 topics
dc_lda <- LDA(dc_dtm, k = 4, control = list(seed = 42))

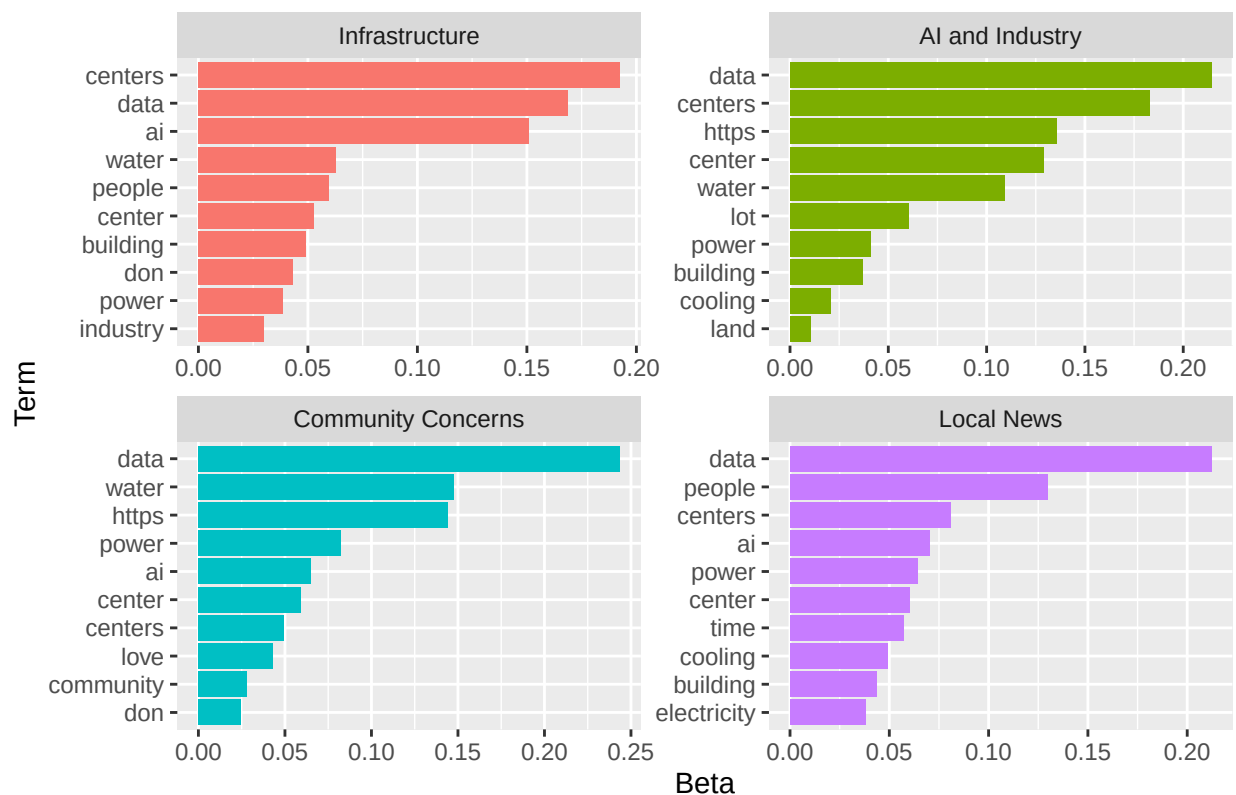
# get top terms per topic
dc_topics <- tidy(dc_lda, matrix = "beta")

dc_top_terms <- dc_topics %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%
  ungroup() %>%
  arrange(topic, -beta) %>%
```

```
mutate(term = reorder_within(term, beta, topic))

# plot with topic labels
ggplot(dc_top_terms, aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free",
             labeller = as_labeller(c("1" = "Infrastructure",
                                       "2" = "AI and Industry",
                                       "3" = "Community Concerns",
                                       "4" = "Local News")))) +
  scale_y_reordered() +
  labs(title = "Top Terms per Topic in Data Center Reddit Discussion",
       x = "Beta", y = "Term")
```

Top Terms per Topic in Data Center Reddit Discussion



5. Consider how LLMs (specifically, their ability to write large amount of text and code with a single prompt) will change the social sciences. Will this technology free up researchers to pursue bigger ideas, or will it lead to “mediocrity at scale?” What do you see as the most important opportunities and challenges related to LLMs in the social sciences? (Please refrain from using your LLM to write your answer - I want to know your thoughts!). **Commit your changes.**

I think LLMs will have a big impact on the social sciences, and honestly I see both sides of it. On one hand, the ability to generate code and write large amounts of text quickly is genuinely useful. Things that used to take hours like writing up methods sections, cleaning data, or figuring out the right syntax for a function can now be done in seconds. This could free up researchers to spend more time on the parts of research that actually require creative thinking, like coming up with interesting questions or interpreting what findings

actually mean for society. But I think there is a real risk of losing something important in the process because a lot of what makes good social science is the deep engagement a researcher has with their data and their subject. When you spend a long time working through your analysis manually, you notice things you might not have expected, you develop intuitions about your data, and you start to really understand the people or communities you are studying. If LLMs are doing most of that work, I worry that researchers will lose that connection to their data and that the work will start to feel more mechanical than meaningful. I also think passion matters a lot in research. The best social science comes from researchers who genuinely care about the questions they are asking. If LLMs make it too easy to produce papers quickly, I think we will see a lot more of what the SocArxiv editors called “AI-generated slop,” work that is technically competent but does not really say anything interesting or important. Engzell and Wilmers found that even their best LLM generated papers struggled with framing and making arguments of broad interest, which makes sense because those are the parts of research that require actually caring about the question. So while I see both sides, I think human-driven research and papers is key to the future of academia and societal advancement in the social science field.